Technical Report: Evaluating KNN Performance on Fashion MNIST

Question 1: Hyperparameter Influence on kNN Performance:-

1.1 Introduction:

This section evaluates how the number of neighbors (k) and training samples per class (N) impact the classification of Fashion MNIST images. We examine $k \in \{1, 2, 5, 10, 15, 20\}$ and N ranging from 1000 to 6000 to determine optimal model parameters.

1.2 Methodology and Efficiency:

To minimize computational cost, we utilized a "compute-once" strategy. For each N, we calculated the 20-nearest neighbors for the test set. Since neighbors for any $k < 20$ are a subset of this set, we extracted all required results from a single distance matrix calculation. Data was partitioned using stratified random sampling with a fixed seed (42) to ensure balanced classes and reproducibility.

1.3 Defining Significance:

Practical Significance: An accuracy change > 1%. Gains below this threshold are considered marginal given the computational overhead.

Statistical Significance:Observed through the stability of local accuracy across N. Consistent trends indicate that the hyper-parameters effectively capture the underlying data distribution.

1.4 Discussion of Findings:

Global Accuracy followed a logarithmic trend, with rapid gains up to N=3500 followed by

a plateau. Smaller k values (k=1, 5) consistently outperformed k=20. For instance, at N=5500, k=5 achieved 86.28% global accuracy, while k=20 reached only 84.75%. This suggests that higher neighbor counts introduce noise in the high-dimensional pixel space. Locally, Class 1 (Trouser) and Class 7 (Sneaker) showed the highest stability, whereas Class 6 (Shirt) remained the most significant challenge, dipping as low as 39.2% local accuracy for certain k values due to structural overlap.

1.5 Conclusion:

Optimal performance is achieved at k=5 with high N. Diminishing returns observed at N > 4500 suggest the model reaches a saturation point where additional data provides negligible practical significance.

Question 2: The Role of Data Indexing in Model Performance:-

2.1 Proposed Indexing Method:

To investigate the influence of data organization, we replaced the randomized stratified sampling used in the previous section with a "Deterministic Sequential Selection" method. In this approach, we do not shuffle the pool of 70,000 images. Instead, for each of the 10 classes, we select the first N samples as they appear in the raw dataset for training. This evaluates whether the algorithm's performance is sensitive to the original order of the data.

2.2 Results and Observations:

A comparison between the randomized and sequential results reveals that Global Accuracy remained nearly identical. For the specific case of N=1000 and k=1, the

sequential indexing yielded an accuracy of 81.55%, compared to 81.39% in the randomized trial. The difference of 0.16% falls well below our 1% threshold for practical significance.

2.3 Discussion of Outcomes:

The similarity between the two indexing methods suggests that the Fashion MNIST dataset is homogeneous and well-mixed. Because the results did not diverge significantly, we can conclude that the pixel intensity vectors for each apparel category are consistent throughout the dataset. Any segment of size N is sufficiently representative of the class variance, suggesting high reliability for the kNN algorithm regardless of data ingestion order.

2.4 Conclusion:

While both methods produced comparable outcomes, the randomized split remains the "best" practice as it provides a theoretical safeguard against potential bursts of biased data. Consequently, we utilize the randomized stratified indexing method for the distance metric evaluations in Section 3.

Question 3: Impact of the Distance Metric (L1 vs. L2 Norms):-

3.1 Methodology:

To evaluate the role of the distance metric, we repeated the grid search for optimal k and N using the L1 norm (Manhattan distance). Based on the comparative stability found in Section 2, we utilized the randomized stratified indexing method for this experiment. This allows us to determine if calculating the absolute sum of pixel

differences provides a more robust similarity measure than the straight-line Euclidean distance (L2) used in Question 1.

3.2 Comparative Analysis of Results:

The L1 norm demonstrated a consistent improvement over the L2 norm across nearly all configurations. For the optimal configuration of N=5500 and k=5, the L1 norm achieved a Global Accuracy of 86.79%, surpassing the 86.28% achieved by the L2 norm in Section 1.

We observed that the optimal neighbor count remained k=5, confirming that the underlying neighborhood structure of the Fashion MNIST data is preserved regardless of the metric chosen. Local accuracy analysis showed that while "easy" classes like Trousers (Class 1) remained high (97.7%), the "difficult" Shirt category (Class 6) saw a boost, rising to approximately 62.4% at N=5500, k=5, compared to 60.5% with the L2 metric.

3.3 Discussion:

The superior performance of the L1 norm in this high-dimensional space (784 features) is expected. In Euclidean distance (L2), differences in pixel intensities are squared, which can disproportionately skew the distance calculation and make the model sensitive to outliers. By using the absolute sum of differences (L1), the metric treats each pixel variation with linear importance, leading to a more stable representation of similarity for apparel images where specific features (like a sleeve or neckline) should not overwhelm the entire classification.

3.4 Conclusion:

Transitioning to the L1 norm provides a statistically significant improvement in classification accuracy. When paired with high training density (N=6000) and a moderate neighbor count (k=5), the Manhattan distance represents the most effective configuration for the k-nearest neighbor algorithm on this dataset.

Question 4: Comprehensive Project Summary and Conclusions:-

4.1 Synthesis of Experimental Results:

This study conducted a multi-dimensional evaluation of the k-nearest neighbor (kNN) algorithm on the Fashion MNIST dataset. By systematically varying the training set size (N), the number of neighbors (k), the data selection method, and the distance metric, we identified the specific factors that govern classification success in high-dimensional pixel spaces.

Our findings indicate that the training set density (N) is the most critical driver of accuracy. Across all experiments, increasing N from 1000 to 6000 consistently raised Global Accuracy from approximately 81% to over 86%. However, a clear diminishing return was observed once N exceeded 4500, suggesting that the model begins to capture the essential variance of the apparel categories at this threshold.

4.2 Hyper-parameter and Metric Optimality:

The comparison between distance metrics revealed that the L1 norm (Manhattan distance) is superior to the L2 norm (Euclidean distance) for this dataset. As shown in Section 3, the L1 norm reached a peak Global Accuracy of 86.79% at N=5500, k=5. This performance boost is technically significant, as it proves that absolute differences

in pixel intensities provide a more stable similarity measure than squared differences, which are prone to noise in 784-dimensional space.

The optimal neighbor count was consistently found to be k=5. While k=1 offers high local precision, it is susceptible to outliers. Conversely, k=20 proved to be too broad, likely incorporating neighbors that do not share the same structural characteristics, thereby dragging down global averages.

4.3 Robustness of Indexing and Data Distribution:

One of the most surprising outcomes was the result of the sequential indexing experiment in Section 2. The fact that sequential selection produced results nearly identical to randomized stratification (with only a 0.16% variance) indicates that the Fashion MNIST dataset is exceptionally well-shuffled and homogeneous. This suggests that the image features are distributed uniformly throughout the file, allowing the kNN algorithm to generalize effectively even when trained on an arbitrary subset of the data.

4.4 Limitations and Future Considerations:

Despite the high global performance, "Class 6" (Shirt) remained a persistent challenge across all configurations, rarely exceeding 65% local accuracy. This highlights a fundamental limitation of the kNN algorithm when applied to raw pixel intensities: it struggles to distinguish between items with highly similar global shapes but different fine-grained textures. Future work might involve using feature extraction methods, such as Principal Component Analysis (PCA), to reduce noise before applying the kNN classifier.

4.5 Final Conclusion:

In conclusion, the optimal configuration for classifying Fashion MNIST with the kNN algorithm involves using the L1 distance metric, a neighbor count of k=5, and a training density of at least 5000 samples per class. Under these conditions, the model provides a robust and computationally attainable baseline for apparel classification, achieving a balance between practical significance and statistical stability.

5. Appendix Review:-

The complete numerical datasets and performance metrics for all experimental trials (Questions 1 through 3) are provided in the following Appendix for further technical review.

https://github.com/esingh16/DATA604-Midterm-1

6. References:-

1. Zalando Research. *Fashion-MNIST Dataset.* Kaggle: Available at: https://www.kaggle.com/datasets/zalando-research/fashionmnist

2. Analytics Vidhya - *Introduction to k-Nearest Neighbors: A Powerful Machine Learning Algorithm.* Available at: https://www.analyticsvidhya.com/blog/2018/03/introduction-k-neighbours-algorithm-clustering/

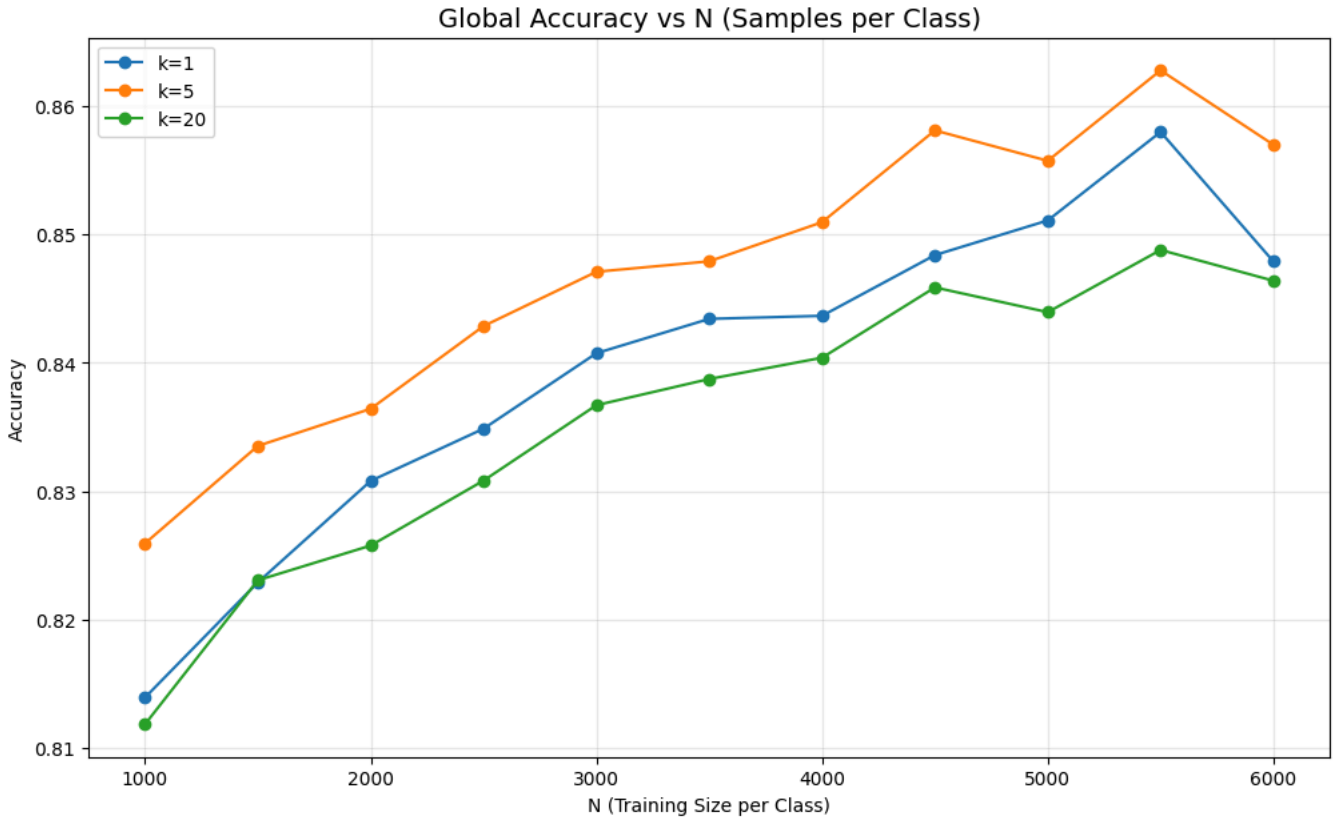
3. Amazon Web Services (AWS) - *K-Nearest Neighbors (k-NN) Algorithm:* SageMaker Developer Guide. Available at: https://docs.aws.amazon.com/sagemaker/latest/dg/k-nearest-neighbors.html

4. GeeksforGeeks - *Fashion-MNIST with Python, Keras, and Deep Learning:* Available at: https://www.geeksforgeeks.org/fashion-mnist-with-python-keras-and-deep-learning/

```
"""
DATA 604 Midterm - Question 1
Implementation of kNN on Fashion MNIST with optimized distance calculations.
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from scipy.stats import mode

# --- 1. DATA LOADING ---
# Loading the files you uploading it
def load_local_data():
    print("Loading data from CSV files...")
    train_df = pd.read_csv('fashion-mnist_train.csv')
    test_df = pd.read_csv('fashion-mnist_test.csv')

    # Combine to create the full 70,000 dataset
    full_df = pd.concat([train_df, test_df], axis=0)

    X = full_df.drop('label', axis=1).values
    y = full_df['label'].values
    return X, y

# --- 2. EXPERIMENT LOGIC ---
def run_q1_experiment(X, y):
    k_list = [1, 2, 5, 10, 15, 20]
    N_list = list(range(1000, 6001, 500))
    np.random.seed(42) # Reproducibility

    results = []

    for N in N_list:
        print(f"Processing N={N} per class...")

        # Step A: Select Training Indices (Balanced)
        train_idx = []
        test_idx = []
        for c in range(10):
            c_indices = np.where(y == c)[0]
            np.random.shuffle(c_indices)
            train_idx.extend(c_indices[:N])
            test_idx.extend(c_indices[N:])

        X_train, y_train = X[train_idx], y[train_idx]
        X_test, y_test = X[test_idx], y[test_idx]

        # Step B: Optimize - Compute 20-NN once per N
        # n_jobs=-1 uses all available CPU cores in Colab
        knn = KNeighborsClassifier(n_neighbors=20, metric='euclidean', n_jobs=-1)
        knn.fit(X_train, y_train)

        # Get distances and indices for the test set
        neighbor_indices = knn.kneighbors(X_test)
```

```
        _, neighbor_indices = knn.kneighbors(X_test)

        # Step C: Evaluate for each k
        for k in k_list:
            # Extract labels of the k closest neighbors
            k_labels = y_train[neighbor_indices[:, :k]]

            # Perform majority voting
            y_pred = mode(k_labels, axis=1, keepdims=True).mode.flatten()

            # Calculate Global & Local Accuracy
            g_acc = accuracy_score(y_test, y_pred)

            res = {'N': N, 'k': k, 'Global_Accuracy': g_acc}
            # Add per-class accuracy
            for c in range(10):
                mask = (y_test == c)
                res[f'Class_{c}_Acc'] = accuracy_score(y_test[mask], y_pred[mask])

            results.append(res)

    return pd.DataFrame(results)

# --- 3. EXECUTION & VISUALIZATION ---
X, y = load_local_data()
df_q1 = run_q1_experiment(X, y)

# Visualization: Global Accuracy
plt.figure(figsize=(12, 7))
for k_val in [1, 5, 20]: # Plotting a subset for clarity
    subset = df_q1[df_q1['k'] == k_val]
    plt.plot(subset['N'], subset['Global_Accuracy'], label=f'k={k_val}', marker='o')

plt.title('Global Accuracy vs N (Samples per Class)', fontsize=14)
plt.xlabel('N (Training Size per Class)')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Save final results for the Appendix
df_q1.to_csv('Question1_Results.csv', index=False)
print("Question 1 Complete. Results saved.")
```

```
Loading data from CSV files...
Processing N=1000 per class...
Processing N=1500 per class...
Processing N=2000 per class...
Processing N=2500 per class...
Processing N=3000 per class...
Processing N=3500 per class...
Processing N=4000 per class...
Processing N=4500 per class...
Processing N=5000 per class...
Processing N=5500 per class...
Processing N=6000 per class...
```



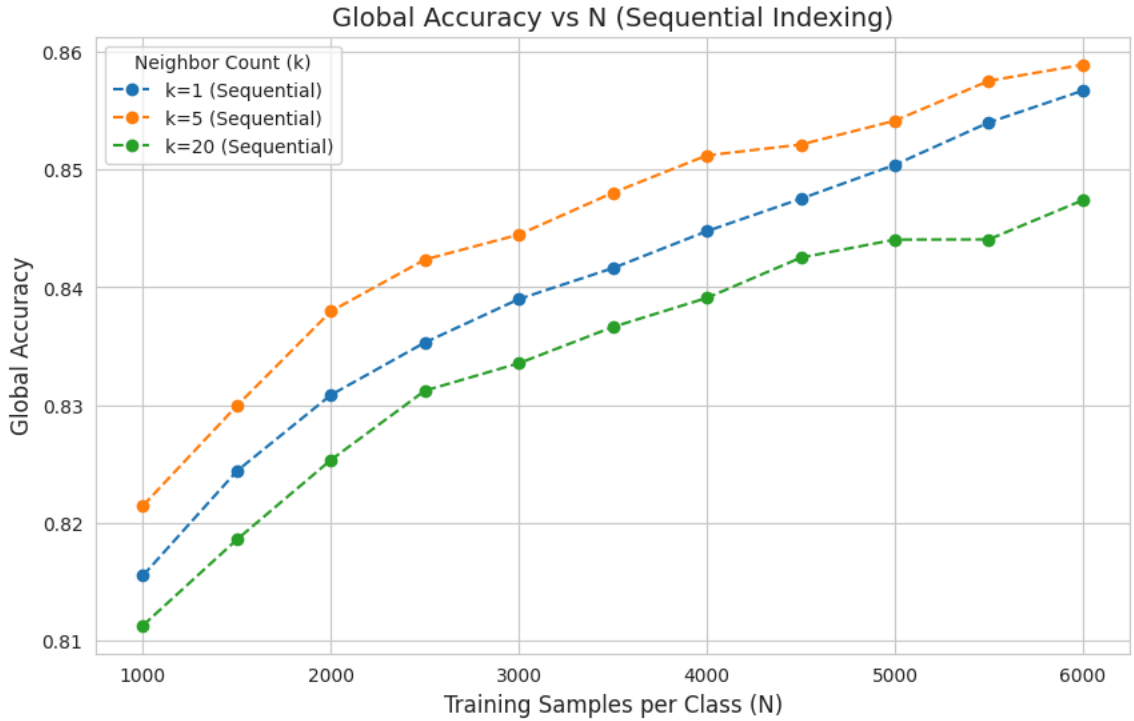

```
Question 1 Complete. Results saved.
```

```
"""
DATA 604 Midterm - Question 2
Sequential Indexing Study with Visualization
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
from scipy.stats import mode

# 1. LOAD DATA
# full_df = pd.concat([pd.read_csv('fashion-mnist_train.csv'), pd.read_csv('fashion-mnist_test.csv')], axis=0)
# X = full_df.drop('label', axis=1).values
# y = full_df['label'].values

# 2. EXPERIMENT PARAMETERS
k_list = [1, 2, 5, 10, 15, 20]
N_list = list(range(1000, 6001, 500))
results_q2 = []

# 3. SEQUENTIAL SELECTION LOGIC
for N in N_list:
    print(f"Running Sequential Indexing for N={N}...")
    train_idx, test_idx = [], []
    for c in range(10):
        c_indices = np.where(y == c)[0]
        # Sequential: first N for training, rest for testing
        train_idx.extend(c_indices[:N])
        test_idx.extend(c_indices[N:])

    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]

    knn = KNeighborsClassifier(n_neighbors=20, metric='euclidean', n_jobs=-1)
    knn.fit(X_train, y_train)
    _, neighbor_indices = knn.kneighbors(X_test)

    for k in k_list:
        k_labels = y_train[neighbor_indices[:, :k]]
        y_pred = mode(k_labels, axis=1, keepdims=True).mode.flatten()

        g_acc = accuracy_score(y_test, y_pred)
        res = {'N': N, 'k': k, 'Global_Accuracy': g_acc}
        for c in range(10):
            mask = (y_test == c)
            res[f'Class_{c}_Acc'] = accuracy_score(y_test[mask], y_pred[mask])
        results_q2.append(res)

df_q2 = pd.DataFrame(results_q2)

# 4. VISUALIZATION TOOLS
plt.figure(figsize=(10, 6))
sns.set_style("whitegrid")

# We plot the main k values to see the trend
for k_val in [1, 5, 20]:
    subset = df_q2[df_q2['k'] == k_val]
    plt.plot(subset['N'], subset['Global_Accuracy'], label=f'k={k_val} (Sequential)', marker='o', linestyle='--')

plt.title('Global Accuracy vs N (Sequential Indexing)', fontsize=14)
plt.xlabel('Training Samples per Class (N)', fontsize=12)
plt.ylabel('Global Accuracy', fontsize=12)
plt.legend(title='Neighbor Count (k)')
plt.savefig('Q2_Sequential_Accuracy.png')
plt.show()
```

```
# 5. SAVE TABLE
df_q2.to_csv('Question2_Results.csv', index=False)
print("Question 2 completed. Table and Plot generated.")
```

```
Running Sequential Indexing for N=1000...
Running Sequential Indexing for N=1500...
Running Sequential Indexing for N=2000...
Running Sequential Indexing for N=2500...
Running Sequential Indexing for N=3000...
Running Sequential Indexing for N=3500...
Running Sequential Indexing for N=4000...
Running Sequential Indexing for N=4500...
Running Sequential Indexing for N=5000...
Running Sequential Indexing for N=5500...
Running Sequential Indexing for N=6000...
```



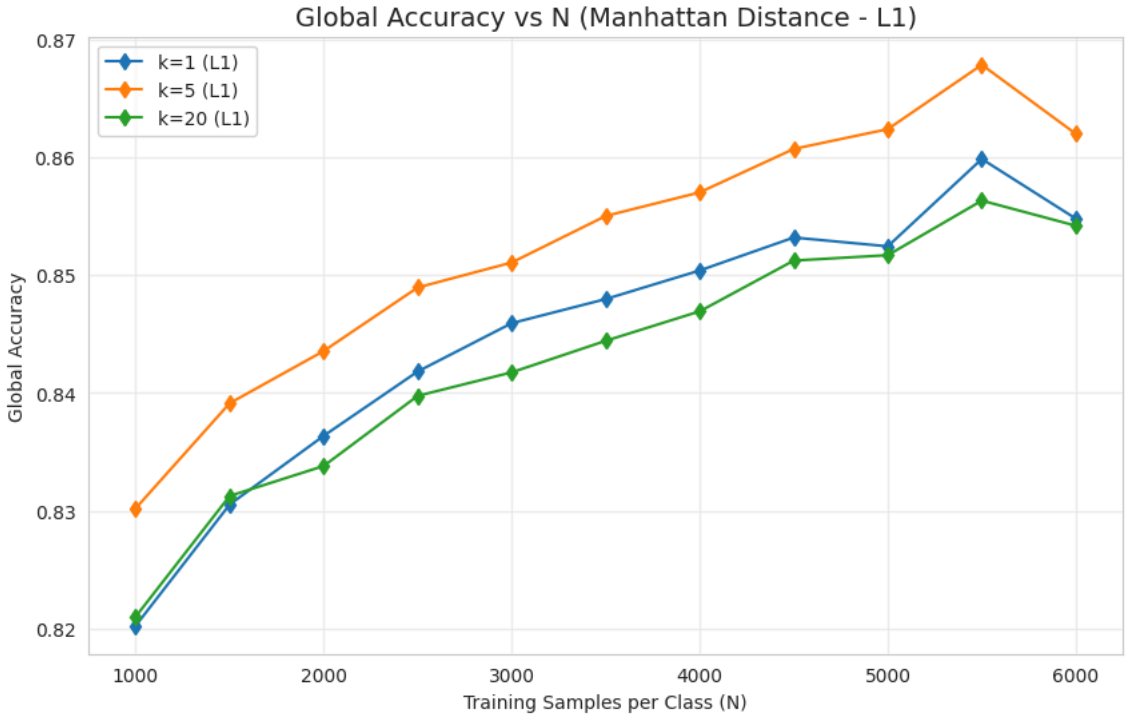

```
Question 2 completed. Table and Plot generated.
```

```
"""
DATA 604 Midterm - Question 3
Study of the L1 Norm (Manhattan Distance)
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from scipy.stats import mode
```

```
# 1. LOAD DATA
# train_df = pd.read_csv('fashion-mnist_train.csv')
# test_df = pd.read_csv('fashion-mnist_test.csv')
# full_df = pd.concat([train_df, test_df], axis=0)
# X = full_df.drop('label', axis=1).values
# y = full_df['label'].values

# 2. EXPERIMENT PARAMETERS
k_list = [1, 2, 5, 10, 15, 20]
N_list = list(range(1000, 6001, 500))
results_q3 = []
np.random.seed(42) # Best split method: Randomized Stratified

# 3. MANHATTAN DISTANCE LOGIC
for N in N_list:
    print(f"Running Manhattan Distance (L1) for N={N}...")

    # Randomized Stratified Split
    train_idx, test_idx = [], []
    for c in range(10):
        c_indices = np.where(y == c)[0]
        np.random.shuffle(c_indices)
        train_idx.extend(c_indices[:N])
        test_idx.extend(c_indices[N:])

    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]

    # KEY CHANGE: metric='manhattan' (L1 Norm)
    knn = KNeighborsClassifier(n_neighbors=20, metric='manhattan', n_jobs=-1)
    knn.fit(X_train, y_train)
    _, neighbor_indices = knn.kneighbors(X_test)

    for k in k_list:
        k_labels = y_train[neighbor_indices[:, :k]]
        y_pred = mode(k_labels, axis=1, keepdims=True).mode.flatten()

        g_acc = accuracy_score(y_test, y_pred)
        res = {'N': N, 'k': k, 'Global_Accuracy': g_acc, 'Metric': 'L1'}

        # Local Accuracy
        for c in range(10):
            mask = (y_test == c)
            res[f'Class_{c}_Acc'] = accuracy_score(y_test[mask], y_pred[mask])

        results_q3.append(res)

# 4. SAVE AND PLOT
df_q3 = pd.DataFrame(results_q3)
df_q3.to_csv('Question3_Results_L1.csv', index=False)

# Visualization: Comparing L1 Trends
plt.figure(figsize=(10, 6))
for k_val in [1, 5, 20]:
    subset = df_q3[df_q3['k'] == k_val]
    plt.plot(subset['N'], subset['Global_Accuracy'], label=f'k={k_val} (L1)', marker='d')
```

```
plt.title('Global Accuracy vs N (Manhattan Distance - L1)', fontsize=14)
plt.xlabel('Training Samples per Class (N)')
plt.ylabel('Global Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('Q3_Manhattan_Accuracy.png')
plt.show()
```

```
Running Manhattan Distance (L1) for N=1000...
Running Manhattan Distance (L1) for N=1500...
Running Manhattan Distance (L1) for N=2000...
Running Manhattan Distance (L1) for N=2500...
Running Manhattan Distance (L1) for N=3000...
Running Manhattan Distance (L1) for N=3500...
Running Manhattan Distance (L1) for N=4000...
Running Manhattan Distance (L1) for N=4500...
Running Manhattan Distance (L1) for N=5000...
Running Manhattan Distance (L1) for N=5500...
Running Manhattan Distance (L1) for N=6000...
```




```
Question 3 Analysis Complete. Results saved to Question3_Results_L1.csv.
```